Chapter-2

Question - 1:

Construct the equivalent RISC-V code of the following C code. Once you have the RISC-V code, identify type of each instruction and encode them accordingly.

```
A[7] = A[2] + A[B[8]] + 10;
B[i] = A[3] - 8;
```

Base addresses of array A and B are in register X_{20} and X_{21} and i is in register X_{22}

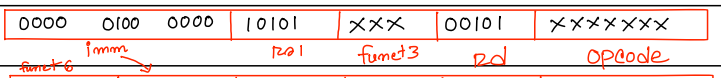
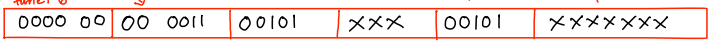
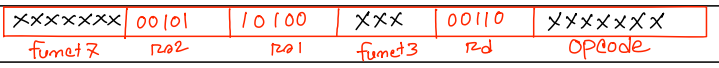
A = X_{20}
B = X_{21}
i = X_{22}

Line-1

```
ld   X5, 64(X21)     // B[8]
Slli X5, X5, 3       // offset calculation (X5 * 2^3)
Add  X6, X20, X5     // physical loc. (Base + offset)
ld   X6, 0(X6)       // A[B[8]]

ld   X5, 16(X20)     // A[2]

Add  X5, X5, X6      // A[B[8]] + A[2]

Addi X5, X5, 10      // A[B[8]] + A[2] + 10

Sd   X5, 56(X20)     // A[7] = A[B[8]] + A[2] + 10
```

Line-2

```
ld   X6, 24(X20)     // A[3]
Addi X6, X6, -8      // A[3] - 8

Slli X7, X22, 3      // offset calc => X22 * 2^3
Add  X7, X7, X21     // Physical Add. calc

Sd   X6, 0(X7)       // B[i] = A[3] - 8
```

No.	Instruction	Type	Encoding
1	ld x_5, 64(x_{21})	I ✓	
2	Slli x_5, x_5, 3	I ✓	
3	Add x_6, x_{20}, x_5	R ✓	
4	ld x_6, 0(x_6)	I	
5	ld x_5, 16(x_{20})	I	
6	Add x_5, x_5, x_6	R	
7	Addi x_5, x_5, 10	I	
8	Sd x_5, 56(x_{20})	S ✓	
9	ld x_6, 18(x_{20})	I	
10	Addi x_6, x_6, -8	I ✓	
11	Slli x_7, x_{22}, 3	I	
12	Add x_7, x_7, 21	R	
13	Sd x_6, 0(x_7)	S	

8 = 1000

+8 = 0 1000

+8 = 0000 0000 1000

1111 1111 0111

+1

-8 = 1111 1111 1000

Encode rest of the instructions yourself.

Question - 2:

Construct the equivalent RISC-V code of the following C code.

```
for (i = 8; i > 0 ; i--) {
  if ( A[i] == i){
    A[2] = A [B[3]] ;
  }
}
```

Base addresses of array A and B are in register X_{20} and X_{21} . Also consider i is in register X_{22}.

A = X_{20}
B = X_{21}
i = X_{22}

```
  Addi  X22, X0, 8
Loop:
   Beq  X22, X0, Exit

  If:
  Slli  X5, X22, 3      // offset calc.
  Add   X5, X5, X20     // Base + offset
  ld    X6, 0(X5)       // X6 = A[i]
  Bne   X6, X22, inc

  ld    X7, 24(X21)     // X7 = B[3]
 Slli   X7, X7, 3       // X7 = X7 x 2^3
 Add    X7, X7, X20     // Base + offset
 ld     X5, 0(X7)       // X5 = A[B[3]]
 sd     X5, 16(X20)     // A[2] = X5

inc:
Addi X22, X22, -1
Beq  X0, X0, Loop

Exit :
```

Question - 3:

Construct the equivalent RISC-V code of the following C code.

```
if ( A[i] < i){
    A[2] = A [B[3]] ;
}
```

Base addresses of array A and B are in register X_{20} and X_{21} . Also consider i is in register X_{22}.

```
If :
slli  x5, x22, 3      // offset calc.
Add   x5, x5, x20     // Base + offset
ld    x6, 0(x5)       // x6 = A[i]
BGE   x6, x22, Exit

ld    x7, 24(x21)     // x7 = B[3]
slli  x7, x7, 3       // x7 = x7 x 2^3
Add   x7, x7, x20     // Base + offset
ld    x5, 0(x7)       // x5 = A[B[3]]
sd    x5, 16(x20)     // A[2] = x5

Exit :
```

A = X_{20}
B = X_{21}
i = X_{22}

Question - 4:

Construct the equivalent RISC-V code of the following C code.

```
if ( A[3] != A[6]){
    if (A[3] == 0) {
        A[3] = A[3] + 2;
    }else{
        A[6] = A[6] / 16;
    }
}else{
    A[6] = A[6] * 8
}
```

Base addresses of array A and B are in register X_{20}

If 1:

```
ld   x28, 24(x20)      // x28 = A[3]
ld   x29, 48(x20)      // x29 = A[6]
Beq  x28, x29, Else1
```

If 2:

```
ld   x5,  24(x20)      // x5 = A[3]
BNE  x5,  x0,  Else2
```

→ A[3] already loaded into x_5 and x_5 is unchanged till now.

```
Addi x5, x5, 2
Sd   x5, 24(x20)       // A[3] = x5 + 2
Beq  x0, x0, Exit
```

Else 2:

```
ld   x5, 48(x20)       // x5 = A[6]
SRLI x5, x5, 4         // x5 = x5 / 2^4
Sd   x5, 48(x20)       // A[6] = x5
Beq  x0, x0, Exit
```

Else 1:

```
ld   x28, 48(x20)      // x28 = A[6]
Slli x28, x28, 3       // x28 = x28 * 8
Sd   x28, 48(x20)      // A[6] = x28
```

Exit:

Question - 5:

Translate the following code written in C programming language into instructions sequence written in RISC-V Assembly. The values in the variables a, b, and c inside the add function are stored in the argument registers $\$X_{10}$, $\$X_{15}$, and $\$X_{12}$ respectively. Also, the values in the variables x and y inside the are stored in the argument registers $\$X_{13}$ and $\$X_{14}$ respectively. The return value is stored in the return register $\$X_{11}$ for both functions.

int max(int x, int y) {
  if(x > y)
  return x;
  else
  return y;
}
	int calc(int a, int b, int c) {
	  return a + max(b,c)
	}

x_{10} x_{15} x_{12} — x_{13} x_{14}

x_{13} < x_{14}

Calc:

```
Addi   Sp, Sp, -8
sd     X1, 0(SP)
Addi   X13, X15, 0      [X13 = X15 + 0]   (b)
Addi   X14, X12, 0      [X14 = X12 + 0]   (c)
Jal    X1, max
Add    X11, X10, X11
ID     X1, 0(SP)
Addi   Sp, Sp, 8
Jalr   X0, 0(X1)
```

X10 a, X11 return from max

Max:

```
Blt    X13, X14, maxElse   (
Addi   X11, X13 0
Beq    X0, X0, ExitMax

maxElse:
Addi   X11, X14, 0

ExitMax:
Jalr   X0, 0(X1)
```

X13 n, X14 y

Quer – 6

Write RISC-V assembly code that checks if the number stored in register $\mathbf{X_{25}}$ is **even** or not. If **even** then store **1** in register $\mathbf{X_{26}}$ otherwise store **0**.

0000 .. .... 0001 = 1

```
Addi  x27, x0, 1
And   x27, x25, x27 // LSB bit required
BNE   x27, x0, else
Addi  x26, x0, 1
Beq   x0, x0, Exit

else:
Addi  x26, x0, 0
Exit:
```

You can also use Slli / Srli to figure it out.

Quer-7

ADD X_{25} , X_{25} , X_0 . Can you make this instruction faster? If yes, Write the updated instruction?

Yes, ADDI x_{25}, x_{25}, 0

0 is hard wired here.

Quer-8

Memory Location	Code	Line Number	Machine Code
	ADDI $\mathbf{X_5}$, X_0 , 5	1	
# 7064	ADDI $\mathbf{X_6}$, X_0 , 1	2	
# 7068	ADDI $\mathbf{X_{25}}$, X_0 , 0	3	
# 7072	Loop: BLT $\mathbf{X_5}$, $\mathbf{X_6}$, loopBreak	4 -3×4 =-12	0 | 000 000 | 00110 | 00101 | XXX | 1000 | 0 | XXXXXXX (imm[12] imm[10:5] rs2 rs1 imm[4:1] imm[11])
# 7076	ADDI X_{25} , X_{25} , 1	5	
#7080	ADDI X_5 , X_5 , -1	6	
	BEQ $\mathbf{X_0}$, $\mathbf{X_0}$, Loop	7	1 | 111 111 | 00000 | 00000 | XXX | 1010 | 1 | XXXXXXX (12 10:5 4:1 11)
	loopBreak:	8 +4×4 =16	

a) What is the value of **PC** while executing line2? Answer:

7064 [2]

b) Fill up the machine codes corresponding to line4 and line7 in the table above. [5]

16 = 0000 0001 0000

+16 = 0 0000 0001 0000 x

(12 11 10 5 4 1)

12 = 0000 0000 1100

+12 = 0 0000 0000 1100

= 1 1111 1111 0011

+ 1

-12 = 1 1111 1111 0100 0x

(12 11 10 5 4 1)

Ques - 9

# Location		Line No.
	Loop :	
# 80000	Slli X_{10}, X_{22}, 3	1
# 80004	Add X_{10}, X_{10}, X_{25}	2
# 80008	Ld X_9, 0(X_{10})	3
# 80012	Bne X_9, X_{24}, Exit	4
# 80016	Addi X_{22}, X_{22}, 1	5
# 80020	Beq X_0, X_0, loop	6
	Exit:	
# 80024	⟶	7

rs1 rs2

SB type instructions are — Beq X_0, X_0, loop

Bne X_9, X_{24}, Exit

For Beq,

We are moving 5 instruction upward.

Hence, $-5 \times 4 = -20$ (upward direction)

$20 = 0000\ 0001\ 0100$

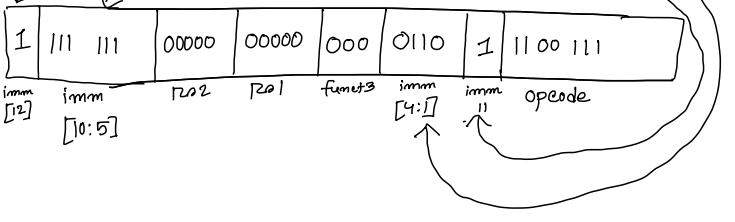
$+20 = 0\ 0000\ 0001\ 0100$

$1\ 1111\ 1110\ 1011$

$+\ 1$

$1\ 1111\ 1110\ 1100$

12 | 11 | 10:5 | 4:1 | 0 x



For Bne,

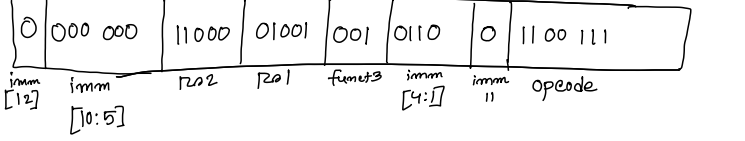
We are moving 3 instructions downward

Hence, $+3 \times 4 = +12$ (downward direction)

$12 = 0000\ 0000\ 1100$

$+12 = 0\ 0000\ 0000\ 1100$

12 | 11 | 10:5 | 4:1 | 0 x



Quez -10

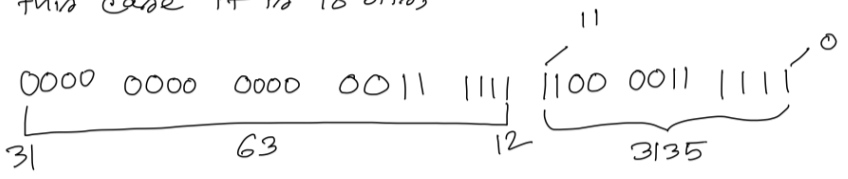
Write necessary RISC-V instructions to store the value $(1111\ 1111\ 0000\ 1111\ 11)_2$ in X20 register.

(18 bits)

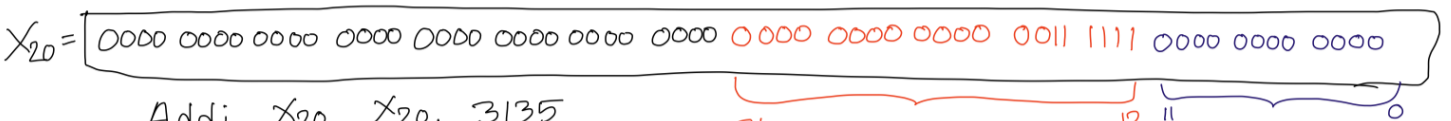
Answer2:

Check if the given number requires more than 12 bits to store.

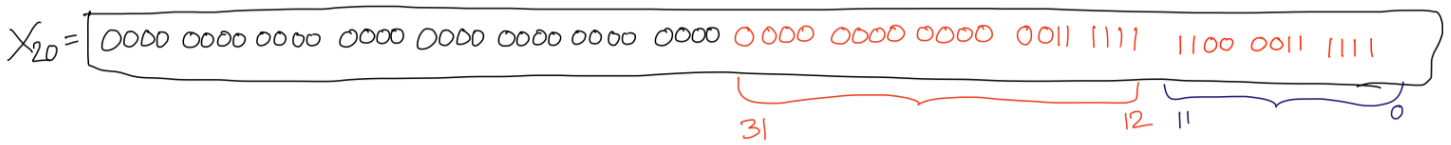
In this case it is 18 bits,



Lui X20, 63



Addi X20, X20, 3135



Quez-12

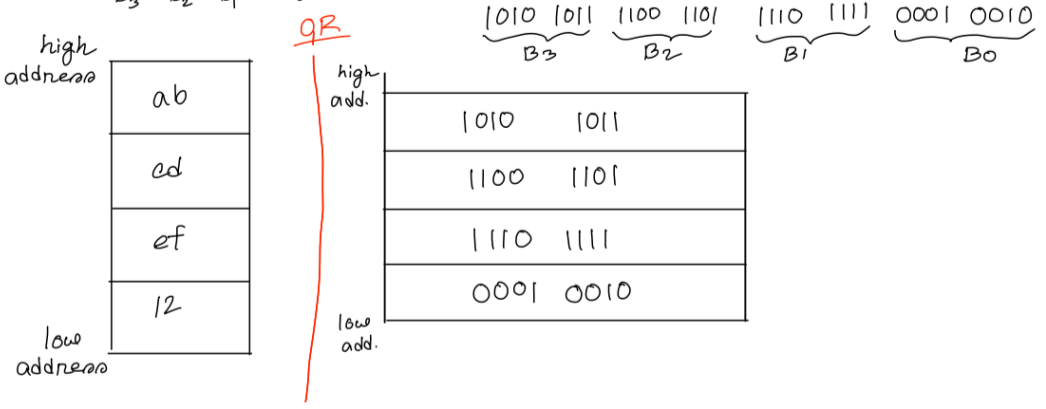
Show how the value 0xabcdef12 would be arranged in memory in RISC-V machine.

Answer3:

RISC-V follows Little endian approach to store data in memory

$\Rightarrow$ $(abcdef12)_{16}$

B_3 B_2 B_1 B_0



Ques - 12

For the RISC-V assembly instructions below, what is the corresponding C/high level statement?

slli x30, x5, 3 $\Rightarrow X_{30} = f \times 8$ add x30, x10, x30 $\Rightarrow X_{30} = Base_A + 8f$ slli x31, x6, 3 $\Rightarrow X_{31} = g \times 8$ add x31, x11, x31 $\Rightarrow X_{31} = Base_B + 8g$ ld x5, 0(x30) $\Rightarrow f = A[f]$ addi x12, x30, 8 $\Rightarrow X_{12} = X_{30} + 8$ ld x30, 0(x12) $\Rightarrow X_{30} = A[f+1]$ add x30, x30, x5 $\Rightarrow X_{30} = A[f+1] + A[f]$ sd x30, 0(x31) $\Rightarrow B[g] = A[f+1] + A[f]$	Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively. Assume that the base address of the Arrays A and B are in registers x10 and x11, respectively.

$B[g] = A[f+1] + A[f]$

Question - 13:

Construct the equivalent RISC-V code **without using the multiplication operation** of the following C code:

```
int n = (a * 19)
```

Assume: n in x21, a in x22 *[Hint: use slli]*

Ans:

```
Slli x5, x22, 4          // x5 = 16a
Slli x6, x22, 1          // x6 = 2a
Add x21, x5, x6          // x21 = 16a + 2a
Add x21, x21, x22        // x21 = 18a + a
```

Question - 14:

Construct the equivalent RISC-V code of the following C code:

```
int n = 0;
for(int i = 5; i <= n; i--) {
  if(A[i] > 0 && A[i] < n){
        A[i] = A[i] - 1;
    }
}
```

Assume : A base address in x20, n in x21, i in x22

Ans:

```
    Add x21, x0, x0
    Addi, x22, x0, 5
Loop: Bgt x22, x21, Exit
    Slli x7, x22, 3
    Add x7, x7, x20
    Ld x6, 0(x7)
    Ble x6, x0, Dec
    Bge x6, x21, Dec
    Addi x6, x6, -1
    Sd x6, 0(x7)
Dec: Addi x22, x22, -1
```

```
        Beq x0, x0, Loop
Exit:
```

Question - 15:

3. Translate the following assembly to equivalent C code.

Assume : A base in x21, B base in x20, i in x22

```
slli x10, x22, 3
add  x10, x10, x20
ld   x5, 0(x10)
addi x5, x5, 1
sd   x5, 24(x21)
```

Ans : A[3] = B[i]+1

Question - 16:

```
bge x22, x21, Exit          \\ 0x015B1A67
slli x5, x22, 3
add x5, x20, x5
ld x10, 0(x5)
addi x10, x10, -1
sd x10, 0(x5)
addi x22, x22, 1
```

The machine code of the first instruction is given beside the instruction. Based on that, deduce

where should the label "Exit" be put?

Ans:

Converting the hex to machine code and putting it in the instruction format of SB type, we get

0	000000	10101	10110	001	1010	0	1100111

The immediate = 000000001010

Therefore, the offset between addresses = 10100 [1 bit left shift] = 20

Offset between lines = 20/4 = 5

So the "Exit" label will go to the instruction: **sd x10, 0(x5)**

Question - 17:

Construct the equivalent RISC-V code of the following C code:

```
int max (int a, int b){
    if(a>b) return a;
    else return b;
}

int main(){
    int x = max(1000, 6000);
}
```

Assume x is in x21.

Ans:

```
Addi x10, x0, 1000
Lui  x11, 1      # Loads 1 << 12 (4096) into x11
Addi x11, x11, 1904  # Adds 1904 to x11 (4096 + 1904 = 6000)
Jal x1, max
Add x21, x10, x0

max:    Ble x10, x11, else
        Beq x0, x0, return
else: Addi x10, x11, x0
return: Jalr x0, 0(x1)
```

Question - 18:

Construct the equivalent **RISC-V assembly** for the following C statement:

A[5] = A[i] * 12 + B[i];

Assume, Base addresses of **A and B are in x20 and x21 respectively. The value of 'i' is in x22 and data size is 4 bytes.**

Solution:

```
slli x5, x22, 2        # x5 = i * 4 (offset for A[i])
add  x5, x20, x5       # x5 = address of A[i]
ld   x6, 0(x5)         # x6 = A[i]


# Multiply A[i] * 12
```

```
# 12 = 8 + 4
slli x7, x6, 3          # x7 = A[i] * 8
slli x8, x6, 2          # x8 = A[i] * 4
add  x7, x7, x8         # x7 = A[i] * 12

slli x9, x22, 2         # offset for B[i]
add  x9, x21, x9        # address of B[i]
ld   x10, 0(x9)         # x10 = B[i]

add  x11, x7, x10       # result = A[i]*12 + B[i]

addi x12, x0, 20        # offset for A[5] (5 * 4)
add  x12, x20, x12
sd   x11, 0(x12)        # store result into A[5]
```

Question - 19:

Assume the value:

0xCAFEBABE12345678

is stored starting at memory address 0x2000. Show the byte order in memory for both little endian and big endian

Solution:

Little Endian:

78	0x2000
56	0x2001
34	0x2002
12	0x2003
BE	0x2004
BA	0x2005

FE	0x2006
CA	0x2007

Big Endian:

CA	0x2000
FE	0x2001
BA	0x2002
BE	0x2003
12	0x2004
34	0x2005
56	0x2006
78	0x2007

Question - 20:

Given the following RISC-V instructions, write their **32-bit machine code** in binary. Put **Don't care(X) where necessary(As opcode, funct3 or funct7 value is not given).**

BEQ x6, x0, target1 (Assume target1 is 32 bytes ahead of the current PC.)

BNE x10, x11, target2 (Assume target2 is 64 bytes before the current PC.)

Solution:

BEQ x6, x0, target1

Offset = 32 bytes → divide by 2 (since LSB[13th bit] always gets discarded)
imm = 32 / 2 = 16 → 000000010000 (12 bits)

Field	Bits	Value
imm[12]	31	0
imm[10:5]	30–25	000001
rs2	24–20	00000

rs1	19–15	00110
funct3	14–12	xxx
imm[4:1]	11–8	0000
imm[11]	7	0
opcode	6–0	xxxxxxx

Machine code: 0 000001 00000 00110 xxx 0000 0 xxxxxxx

BNE x6, x0, target2

Offset = -64 bytes → divide by 2 (since LSB[13th bit] always gets discarded)

imm = -64 / 2 = -32 → 12-bit 2's complement: 111111100000

Field	**Bits**	**Value**
imm[12]	31	1
imm[10:5]	30–25	111110
rs2	24–20	00000
rs1	19–15	00110
funct3	14–12	xxx
imm[4:1]	11–8	0000
imm[11]	7	1
opcode	6–0	xxxxxxx

Machine code: 1 111110 00000 00110 xxx 0000 1 xxxxxxx

Question - 21:

Translate the following **C code** into **RISC-V assembly**. You must use the **LUI instruction**.

int x = 0x12345083;

int y = 25;

int z;

z = x + y;

Solution:

lui x5, 0x12345   # load upper 20 bits of x

addi x5, x5, 0x83   # add lower 12 bits

addi x6, x0, 25   # y = 25

add x7, x5, x6   # z = x + y

Question - 22:

Translate the following **C code** into **RISC-V assembly**.

```
int multiply_add(int a, int b) {
   int c;
   c = a * 4;
   return c + b;
}

int main() {
   int x = 6;
   int y = 9;
   int z;

   z = multiply_add_stack(x, y);
}
```

Assume:

x → x20
y → x21
z → x22
Function arguments → x10, x11
Return value → x10
Stack pointer → x2
a → x13
b → x14
c → x15

NB: Store and restore x13-x15 in stack during procedure

Solution:

```
# main function
addi x20, x0, 6          # x = 6
```

```
addi x21, x0, 9          # y = 9

addi x10, x20, 0         # pass x as argument a
addi x11, x21, 0         # pass y as argument b
jal x1, multiply_add     # call leaf procedure

addi x22, x10, 0         # z = return value


multiply_add:
   addi x2, x2, -24      # allocate 24 bytes on stack for x13, x14, x15
   sd   x13, 0(x2)       # save x13 (a) on stack
   sd   x14, 8(x2)       # save x14 (b) on stack
   sd   x15, 16(x2)      # save x15 (c) on stack

   addi x13, x10, 0      # move argument a to x13
   addi x14, x11, 0      # move argument b to x14
```